

## 简介

本应用手册具体介绍了 FR30xx 中 FlashDB 的功能实现与使用方法。

旨在减少研发工程师的开发时间，提高效率，缩短项目周期。

## 目录

|                                  |    |
|----------------------------------|----|
| 1. FlashDB 模块简介 .....            | 2  |
| 2. 配置说明 .....                    | 3  |
| 2.1. FDB 配置 .....                | 3  |
| 2.2. FAL 配置 .....                | 3  |
| 2.2.1. FAL_FLASH_DEV_TABLE ..... | 4  |
| 2.2.2. FAL_PART_TABLE .....      | 5  |
| 3. 用户接口说明 .....                  | 6  |
| 3.1. flashdb_init .....          | 6  |
| 3.2. flashdb_set .....           | 6  |
| 3.3. flashdb_get .....           | 6  |
| 3.4. flashdb_get_length .....    | 6  |
| 3.5. flashdb_del .....           | 7  |
| 4. 参考示例 .....                    | 8  |
| 5. 变更历史 .....                    | 10 |
| 6. 联系信息 .....                    | 11 |

## 1. FlashDB 模块简介

FlashDB 是一款超轻量级的嵌入式数据库，专注于提供嵌入式产品的数据存储方案。与传统的基于文件系统的数据库不同，FlashDB 结合了 Flash 的特性，具有较强的性能及可靠性。并在保证极低的资源占用前提下，尽可能延长 Flash 使用寿命。FlashDB 提供两种数据库模式：

- 键值数据库：是一种非关系数据库，它将数据存储为键值（Key-Value）对集合，其中键作为唯一标识符。KVDB 操作简洁，可扩展性强。
- 时序数据库：时间序列数据库（TSDB），它将数据按照时间顺序存储。TSDB 数据具有时间戳，数据存储量大，插入及查询性能高。

FR30xx 工程使用的是键值数据库，详细使用说明见后续章节。

## 2. 配置说明

### 2.1. FDB 配置

FlashDB 使用时，可以通过 `fdb_cfg.h` 对其进行功能配置，该文件模板位于 `\components\modules\FlashDB\flashdb\inc` 目录下，可以配置 FlashDB 的基础功能，该部分实际应用时，无需变动。具体配置详情如下：

`FDB_USING_KVDB`

使用 KVDB 功能

`FDB_USING_FAL_MODE`

使用 FAL 模式，FAL 里的分区用于存储数据库。该模式下，FlashDB 直接操作 Flash，所以性能较好

`FDB_WRITE_GRAN`

Flash 写粒度，单位为 bit。默认配置为 8

`FDB_DEBUG_ENABLE`

使能调试信息输出，默认不打开

### 2.2. FAL 配置

FAL（Flash Abstraction Layer）即 Flash 抽象层，主要负责对 flash 读写进行封装，在 `fal_cfg.h` 中进行基础配置，该头文件位于 `\inc` 文件夹中。该文件定义了片上 flash 类型及 flash 存储偏移及大小；在 `fal_flash_port.c` 中对内置 flash 的相关参数进行配置，一般无需改动。

### 2.2.1. FAL\_FLASH\_DEV\_TABLE

Flash 设备列表，定义 FR30xx 系列 Flash 的总容量，块/扇区大小，读写擦除 api 等参数。  
设备表部分一般无需改动。

设备表示例：

```
1. //static sfud_flash_t sfud_dev = NULL;
2. const struct fal_flash_dev onchip_flash =
3. {
4.     .name      = "flashdb_onchip",
5.     .addr      = 0,
6.     .len       = 2 * 1024 * 1024,
7.     .blk_size  = 4*1024,
8.     .ops       = {init, read, write, erase},
9.     .write_gran = 16
10. };

1. /* ===== Flash device Configuration ===== */
2. extern const struct fal_flash_dev onchip_flash;
3.
4. /* flash device table */
5. #define FAL_FLASH_DEV_TABLE \
6. { \
7.     &onchip_flash, \
8. }
```

- “flashdb\_onchip”：Flash 设备的名字。
- 0：对 Flash 操作的起始地址。
- 2\*1024\*1024：Flash 的总大小（2MB）。
- 4\*1024：Flash 块/扇区大小（4K）
- {init, read, write, erase}：Flash 的操作函数
- 16：设置写粒度，单位 bit

### 2.2.2. FAL\_PART\_TABLE

定义 Flash 分区表，每个 Flash 设备可以有 N 个分区，这些分区的集合就是分区表。分区表中的 flash 地址偏移及大小，需要用户配置选择。

分区表示例：

```
1. /* ===== Partition Configuration ===== */
2. #ifdef FAL_PART_HAS_TABLE_CFG
3. /* partition table */
4. #define FAL_PART_TABLE
5. \
6. {FAL_PART_MAGIC_WORD, "FlashEnv", "flashdb_onchip", 2*1000*1024, 12*1024, 0}, \
7. }
8. #endif /* FAL_PART_HAS_TABLE_CFG */
```

上面这个分区表详细描述信息如下：

- “FlashEnv”：分区名
- “flashdb\_onchip”：关联的 flash 设备名
- 2\*1000\*1024：偏移地址
- 12\*1024：大小

需注意以下几点：

- 分区名保证不能重复
- 关联的 Flash 设备务必已经在 Flash 设备表中定义好，并且名称一致
- 分区的起始地址和大小不能超过 Flash 设备的地址范围
- 分区的起始地址需要和用户代码区分开，不能有重叠，建议放到 flash 尾部

### 3. 用户接口说明

#### 3.1. flashdb\_init

int flashdb\_init(void)

初始化 KVDB，配置加锁和解锁函数，用户使用 FlashDB 前，需调用 flashdb\_init 进行初始化。

#### 3.2. flashdb\_set

fdb\_err\_t flashdb\_set(uint32\_t key, uint8\_t \*value, uint32\_t length)

| 参数     | 描述     |
|--------|--------|
| tkey   | KV 的名称 |
| value  | 写入数据指针 |
| length | 写入长度   |

#### 3.3. flashdb\_get

size\_t flashdb\_get(uint32\_t key, uint8\_t \*value, uint32\_t length)

| 参数     | 描述       |
|--------|----------|
| tkey   | KV 的名称   |
| value  | 读取数据存放地址 |
| length | 读取长度     |

返回值为实际读取长度

#### 3.4. flashdb\_get\_length

size\_t flashdb\_get\_length(uint32\_t key)

| 参数   | 描述     |
|------|--------|
| tkey | KV 的名称 |

获取 FlashDB 中对应键值的实际存储长度

### 3.5. flashdb\_del

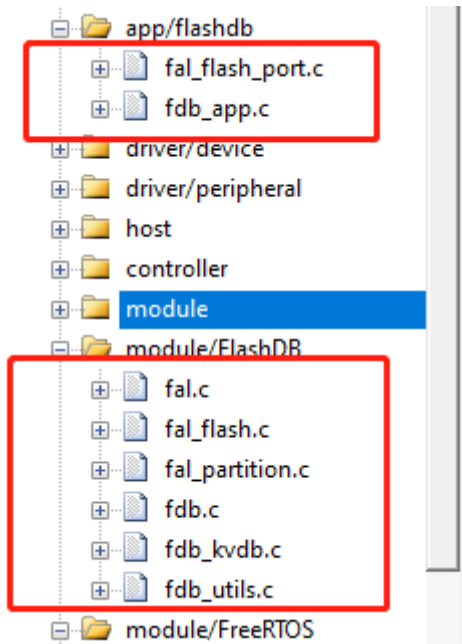
fdb\_err\_t flashdb\_del(uint32\_t key)

| 参数   | 描述     |
|------|--------|
| tkey | KV 的名称 |

删除 FlashDB 中对应键值数据

## 4. 参考示例

- 用户工程需要添加 FlashDB 模块及 fal\_flash\_port.c 和 fdb\_app.c 函数，详细文件见下图。



- 在 fdb\_app.h 中添加自定义的 key 类型，从 FDB\_KEY\_USER\_BASE 地址开始，其他已定义地址段为内部使用

```
#define FDB_KEY_BT_LINKKEY          0x00010000  //!< bt link key storage index
#define FDB_KEY_CONTROLLER_INFO     0x00010001  //!< controller information storage index
#define FDB_KEY_BTDM_LIB_BASE       0x00020000  //!< base index used to store host data
#define FDB_KEY_USER_BASE           0x00030000
#define FDB_KEY_TEST                 (FDB_KEY_USER_BASE + 1)
```

- 调用 flashdb\_init()初始化 FlashDB
- 调用 FlashDB 进行读写操作

```
uint8_t value[2] = {3,4};
uint8_t rd_val[2] = {0};
uint16_t len = 0;
printf("flashdb get length\r\n");
len = flashdb_get_length(FDB_KEY_TEST);
printf("FDB KEY TEST, len = %d\r\n",len);
flashdb_get(FDB_KEY_TEST,rd_val,2);
printf("rd val first time = %d,%d\r\n",rd_val[0],rd_val[1]);
printf("flashdb write new value\r\n");
```

```
flashdb_set(FDB_KEY_TEST,value,2);  
flashdb_get(FDB_KEY_TEST,rd_val,2);  
printf("rd val second time= %d,%d\r\n",rd_val[0],rd_val[1]);  
printf("delete key\r\n");  
flashdb_del(FDB_KEY_TEST);
```

实际输出结果如下：

```
flashdb get length  
FDB KEY TEST, len = 0  
rd val first time = 0,0  
flashdb write new value  
rd val second time= 3,4  
delete key ~
```

## 5. 变更历史

| 版本号  | 日期        | 更新内容 |
|------|-----------|------|
| V1.0 | 2023.9.14 | 首版   |
|      |           |      |
|      |           |      |

## 6. 联系信息

公司：上海富芮坤微电子有限公司

地址：中国(上海)自由贸易试验区碧波路 912 弄 8 号 501-A 室

电话：+86-21-5027-0080

Website: [www.freqchip.com](http://www.freqchip.com)

Sales Email: [sales@freqchip.com](mailto:sales@freqchip.com)

本文档的所有部分，其著作权归上海富芮坤微电子有限公司（简称富芮坤）所有，未经富芮坤授权许可，任何个人及组织不得复制、转载、仿制本文档的全部或部分。富芮坤保留在不另行通知的情况下随时对产品或本文档进行更改、修正、增强的权利。购买者应在订购前获得富芮坤产品的最新相关资料。